Tiny-LM with Jacobi-Forced Micro-Encoder for Collaborative Inference between Edge SLMs and Split-LLMs

Brendon Pedro, Sirapop Nuannimnoi, Panuwat Saetee, Hojjat Baghban, *Member, IEEE,* Karin Honda, and Ching-Yao Huang

***Abstract*—As large language models (LLMs) evolve, deploying them solely in the cloud or compressing them for edge devices has become inadequate due to concerns about latency, privacy, cost, and personalization. Hybrid inference paradigms that combine cloud-based LLMs with edge-deployed small language models (SLMs) have emerged as a promising solution, enabling efficient and privacy-aware processing. However, a key challenge in such systems lies in designing lightweight and adaptive routing mechanisms that can accurately select the appropriate model for each query under resource constraints. In this paper, we propose an intelligent routing framework based on a Tiny Language Model (TLM) that performs fast and efficient query understanding to guide inference selection between SLMs and Split-LLMs in edge–cloud collaborative environments. The proposed approach enables low-latency decision-making while accounting for system-level constraints such as communication overhead, computational cost, and response time. Unlike existing routing methods that rely on heavy models or complex scoring mechanisms, our framework is lightweight, scalable, and adaptable to heterogeneous model deployments without requiring costly retraining. Extensive experiments demonstrate that the proposed method significantly reduces inference cost and latency while maintaining competitive response quality, highlighting its effectiveness for real-time and resource-constrained applications. The results indicate that TLM-based routing provides a practical and scalable solution for efficient LLM–SLM collaboration in modern deployment scenarios.**

***Index Terms*—Large Language Models, Small Language Models, LLM-SLM collaboration, Collaborative Inference**

I. Introduction

LARGE language models (LLMs) have demonstrated remarkable capabilities across a wide range of natural language processing tasks, leading to their rapid adoption in high-stakes domains such as finance and healthcare. In these settings, model outputs must be not only semantically accurate but also privacy-preserving and contextually appropriate. Consequently, practical LLM pipelines require robust mechanisms for identifying domain-relevant information and safeguarding sensitive data [1], [2]. Privacy-preserving architectures are particularly critical in regulated environments (e.g., HIPAA-compliant systems) [3], where governed representations and anonymization pipelines are employed to mitigate re-identification risks. To address these challenges, Split-LLM architectures [5]–[7] have emerged as a promising solution, partitioning model execution between client-side and server-side components such that sensitive data remains local while still leveraging the capabilities of powerful remote models.

Despite these advancements, the high computational and monetary cost of LLM inference—particularly on a per-token basis—renders the use of large models inefficient for simple queries. Small language models (SLMs) [8], [9], in contrast, offer lower latency, reduced energy consumption, and improved deployability on edge devices, making them attractive for lightweight tasks. However, this efficiency often comes at the expense of reduced accuracy and limited reasoning capabilities. This inherent trade-off has motivated the development of hybrid inference paradigms [11]–[18], which dynamically allocate inference workloads based on query complexity. In such frameworks, routine queries are handled locally by SLMs, while complex or ambiguous queries are escalated to more powerful LLMs deployed in cloud or Split-LLM environments. This collaborative approach has the potential to significantly reduce latency, bandwidth usage, and operational costs while maintaining high response quality.

A key challenge in realizing effective LLM–SLM collaboration lies in designing routing mechanisms that can accurately and efficiently select the appropriate model under diverse constraints. Early heuristic-based methods have been largely superseded by learning-based approaches that formulate routing as a decision problem. Recent studies explore a wide range of strategies, including token-level routing for fine-grained control [11], communication-efficient inference for distributed systems [12], and system-level throughput optimization via pipelined decoding [13]. In parallel, advanced routing frameworks leverage signals such as marginal performance gain [14], model capability representations [15], interpretable ability–difficulty modeling [16], and human preference alignment [17], [18].

Despite these advances, existing routing methods still face important practical limitations, particularly in real-time and resource-constrained environments. Current approaches often struggle to jointly optimize accuracy and system-level efficiency, while remaining adaptable to evolving model pools and deployment conditions. Moreover, routing decisions must account not only for prediction quality but also for communication overhead, memory constraints, and response latency, especially in edge–cloud and Split-LLM collaborative

This paper was produced by the IEEE Publication Technology Group. They are in Piscataway, NJ.

Manuscript received April 19, 2021; revised August 16, 2021.

0000–0000/00$00.00 © 2021 IEEE

settings. Designing routing mechanisms that are both adaptive and lightweight therefore remains an open challenge.

To address this gap, we propose an intelligent routing framework based on a Tiny Language Model (TLM), which performs fast and lightweight query understanding to guide inference selection between SLMs and Split-LLMs in a collaborative environment.

The main contributions of this paper are as follows:

- **A lightweight TLM-based routing framework for hybrid LLM–SLM inference:** We propose a novel routing architecture that leverages a Tiny Language Model (TLM) to perform fast and efficient query understanding, enabling dynamic selection between edge-deployed SLMs and cloud-based Split-LLMs. Unlike existing routing approaches that rely on heavy models or complex scoring pipelines, the proposed method achieves low-latency decision-making suitable for real-time deployment.
- **Joint optimization of accuracy and system-level efficiency in edge–cloud environments:** We design the routing mechanism to explicitly consider not only prediction quality but also system constraints, including communication overhead, inference latency, and computational cost. This makes the framework particularly suitable for resource-constrained and privacy-sensitive settings, such as Split-LLM architectures and edge–cloud collaboration.
- **Adaptive and scalable routing without reliance on costly retraining:** The proposed framework supports flexible integration of heterogeneous models (SLMs and Split-LLMs) and adapts to varying query complexity without requiring expensive retraining or large-scale supervision, improving scalability compared to existing learned routing methods.
- **Comprehensive evaluation demonstrating efficiency–performance trade-offs:** We conduct extensive experiments to evaluate routing accuracy, latency, and cost efficiency, showing that the proposed method significantly reduces inference overhead while maintaining competitive response quality compared to conventional LLM-only and existing routing-based approaches.

The remainder of this paper is organized as follows. Section II reviews related work on Split-LLMs and LLM–SLM collaborative inference. Section III presents the system model and problem formulation. Section IV describes the proposed framework. Section V details the experimental setup, and Section VI discusses the results and analysis. Finally, Section VII concludes the paper and outlines future research directions.

II. Related Work

A. Split-LLM

The increasing deployment of LLM-based services has raised significant privacy and security challenges, particularly when sensitive user data is processed during inference. In a split computing environment, a model can be divided into S total splits, thus called "Split-LLM", with a strategic subset offloaded to client or edge devices to leverage local computation. This paradigm is closely related to split learning and collaborative inference frameworks that distribute model execution across multiple nodes [5]–[7]. Specifically, allocating a minority of the model partitions (fewer than $S/2$) to the edge node allows the system to extract high-level features and compress intermediate representations before transmitting them over the Wide Area Network (WAN).

Some studies incorporate stronger privacy mechanisms into Split-LLM systems. For example, Mai et al. [10] further enhance privacy through Split-N-Denoise (SnD), which adds calibrated noise to transmitted embeddings combined with client-side denoising to preserve model performance under differential privacy guarantees.

B. LLM-SLM Collaborative Inference

The LLM–SLM collaborative paradigm integrates the complementary strengths of large language models (LLMs) and small language models (SLMs) through hierarchical cooperation across cloud and edge environments, aiming to balance accuracy, latency, and resource efficiency. At the finest granularity, W. Zheng et al. [11] propose CITER, a token-level routing framework that dynamically assigns non-critical tokens to SLMs while reserving critical tokens for LLMs. The routing policy is optimized via reinforcement learning with joint quality–cost objectives, enhanced by preference-based learning and efficient reward estimation, achieving significant cost reduction without degrading output quality.

Extending collaboration to distributed settings, C. Zheng et al. [12] address communication bottlenecks in speculative decoding by introducing Truncated Sparse Logits Transmission (TSLT), which transmits only a subset of logits while preserving decoding correctness. Combined with multi-candidate token-tree verification, the approach improves throughput and reduces latency, highlighting the importance of communication-efficient collaboration. At the system level, Xu et al. [13] propose DiP-SD, a distributed pipelined speculative decoding framework that jointly optimizes batching, draft lengths, and pipeline scheduling for multi-user edge environments. By overlapping device-side drafting with server-side verification, DiP-SD improves resource utilization and achieves up to 17.89× throughput gains over conventional decoding.

Beyond system optimization, recent works focus on learning-based routing strategies. For example, Luo et al. [14] introduce RouteLMT, which formulates routing as a budget allocation problem and uses marginal gain as the decision signal, predicted directly from SLM representations. This enables superior quality–cost trade-offs without external predictors. Wang et al. [15] also propose ICL-Router, which encodes model capabilities as in-context vectors and decouples capability profiling from routing, enabling scalable integration of new models while achieving state-of-the-art performance. Similarly, Song et al. [16] present IRT-Router, which models LLMs and queries using Item Response Theory, explicitly capturing model ability and query difficulty. This principled formulation enables transparent, cost-aware routing and achieves strong performance–cost trade-offs across 20 LLMs and 12 datasets. In contrast, Ong et al. [17] propose RouteLLM, a

TABLE I
COMPARISON OF LLM–SLM COLLABORATIVE INFERENCE AND ROUTING APPROACHES

Method	Granularity	System-Aware	Learning-Based	Lightweight	Adaptive	Split-LLM Support
CITER [11]	Token-level	✗	✓	✗	✗	✗
TSLT [12]	Sequence-level	✓(Comm.)	✗	✗	✗	✗
DiP-SD [13]	System-level	✓(Throughput)	✗	✗	✗	✗
RouteLMT [14]	Query-level	✗	✓	✓	✓	✗
ICL-Router [15]	Query-level	✗	✓	✗	✓	✗
IRT-Router [16]	Query-level	✓(Cost)	✓	✗	✓	✗
RouteLLM [17]	Query-level	✓(Cost)	✓	✗	✓	✗
Arch-Router [18]	Query-level	✗	✓	✓	✓	✗
Ours (TLM-based)	Query-level	✓(Full)	✓	✓	✓	✓

TABLE II
SUMMARY OF NOTATIONS

Notation	Description
$\mathcal{Q}$	Set of input queries
$q \in \mathcal{Q}$	A user query
$\mathcal{M}$	Set of available models
$M_k \in \mathcal{M}$	The k-th model (SLM or LLM/Split-LLM)
K	Total number of models in $\mathcal{M}$
$R(\cdot)$	Routing function mapping queries to models
$R(q)$	Selected model for query q
$U(M_k, q)$	Expected utility (response quality) of model M_k for query q
$C(M_k, q)$	Cost of using model M_k for query q
λ	Trade-off parameter between utility and cost
$\mathbb{E}_{q\sim\mathcal{Q}}[\cdot]$	Expectation over query distribution
$f_\theta(\cdot)$	TLM-based scoring function
θ	Parameters of the Tiny Language Model (TLM)
$f_\theta(q, M_k)$	Suitability score of model M_k for query q

preference-driven routing framework that predicts whether a strong model significantly outperforms a weak one for a given query. By learning from human preference data, the method reduces inference cost by 2–3.6× while retaining approximately 87–95% of strong-model performance, with minimal routing overhead and good generalization to unseen models. Finally, Tran et al. [18] introduce Arch-Router, a preference-aligned routing framework that maps queries to natural language-defined routing policies based on a Domain–Action taxonomy. By decoupling route selection from model assignment, the framework enables flexible integration of new models without retraining. A compact 1.5B generative router, trained via a two-phase data pipeline, achieves state-of-the-art performance with 93.17% accuracy while being over 28× faster than competing models, demonstrating the effectiveness of aligning routing decisions with human-defined preferences.

III. PROBLEM STATEMENT AND FORMULATION

In hybrid LLM–SLM systems, the objective is to efficiently process user queries by dynamically selecting the most appropriate model under performance and resource constraints. Consider a set of available models $\mathcal{M} = \{M_1, M_2, ..., M_K\}$, where each model may correspond to a small language model (SLM) deployed on edge devices or a large language model (LLM) (or Split-LLM) hosted in the cloud.

Given an input query $q \in \mathcal{Q}$, the goal is to determine a routing function

$$R : \mathcal{Q} \to \mathcal{M}, \tag{1}$$

which assigns q to the most suitable model $M_k \in \mathcal{M}$ for inference.

We consider an edge–cloud collaborative environment consisting of:

- **Edge device:** hosts one or more lightweight SLMs capable of fast, low-cost inference.
- **Cloud server:** hosts powerful LLMs or Split-LLMs that provide higher accuracy at increased computational and communication cost.
- **Routing module:** a lightweight decision model (TLM) deployed at the edge, responsible for selecting the inference path.

For each query q, the system can either:

- process q locally using an SLM, or
- offload q (or part of it, in Split-LLM settings) to the cloud for LLM inference.

Each model M_k is associated with:

- an expected response quality $U(M_k, q)$,
- an inference cost $C(M_k, q)$, which may include computation, communication, and latency.

The routing decision must balance this trade-off. Ideally, we aim to maximize overall utility while minimizing system cost:

$$\max_{R} \ \mathbb{E}_{q\sim\mathcal{Q}} \left[U(R(q), q) - \lambda \cdot C(R(q), q) \right], \tag{2}$$

where λ controls the importance of cost relative to performance.

Designing such an effective routing function $R(\cdot)$ is nontrivial due to several practical constraints:

- **Lightweight requirement:** The router must operate with minimal latency and computational overhead, especially when deployed on edge devices.
- **Query understanding:** Accurate routing requires capturing query complexity, intent, and uncertainty.
- **System constraints:** Decisions must account for communication overhead, memory limitations, and response latency.
- **Adaptability:** The routing mechanism should generalize across diverse queries and remain effective as the model pool evolves.

Therefore, to address these challenges, the proposing routing framework based on a Tiny Language Model (TLM) should approximate the routing function as:

$$R(q) = \arg \max_{M_k \in \mathcal{M}} f_\theta(q, M_k), \tag{3}$$

where $f_\theta(\cdot)$ is a lightweight scoring function learned by the TLM to estimate the suitability of each model for the given query. The TLM performs efficient query representation and decision-making at the edge, enabling fast routing with minimal overhead while maintaining high-quality inference through selective offloading to more powerful models.

IV. Proposed Solution

.

V. Experimental Setup

This section explains our system implementation and experiment settings, including the computational resources, datasets, models, performance metrics, and baseline methods.

A. System Implementation and Environment

To evaluate the proposed intelligent router and its integration with the Split-LLM architecture, we established a testbed representing an edge-cloud continuum. The edge environment is hosted on an edge server, which is equipped with an Intel Xeon W-2123 processor, 64GB of system memory, and two NVIDIA GeForce RTX 3090 GPUs with 24 GB of VRAM each. It operates on Ubuntu 20.04.6 LTS and is responsible for executing the client-side Split-LLM computation. The cloud environment is hosted on a dedicated server, which is equipped with ...

B. Tasks and Datasets

We evaluate the proposed intelligent router on the SimpleQA dataset [19] with pre-computed inference results in LLMRouterBench [20]. SimpleQA consists of 4,326 short factual questions spanning a wide range of real-world topics. Compared with larger multi-task benchmarks such as MMLU-Pro, SimpleQA offers a computationally efficient yet sufficiently diverse evaluation setting for studying query-level routing decisions. Since SimpleQA is already directly integrated into LLMRouterBench, routing labels can be automatically constructed without repeatedly executing every computationally expensive pair of SLMs and Split-LLMs, enabling efficient and reproducible experimentation.

C. Performance Metrics

To comprehensively evaluate the proposed method, we measure system performance across several dimensions including latency, throughput, operational cost, communication overhead, and resource utilization.

1) Latency Metrics: Latency is a critical performance indicator for interactive LLM services. Following standard evaluation practices for autoregressive inference systems, we measure the *Time-to-First-Token* (TTFT), which represents the time required to generate the first output token after receiving the input prompt. This metric captures the latency of the prefill phase, including prompt processing on the edge device, activation transmission to the cloud, and server-side computation.

We also measure the *Time-Per-Output-Token* (TPOT), defined as the average time required to generate each subsequent token during the decoding phase. TPOT reflects the efficiency of token generation once the KV cache has been initialized and includes computation and communication costs during distributed decoding.

The overall end-to-end latency for generating N output tokens is computed as

$$T_{total} = TTFT + (N-1) \cdot TPOT \tag{4}$$

In addition, since the proposed framework targets serverless cloud environments, we measure the additional latency introduced by *cold-start events*. Cold-start latency occurs when a serverless function must be initialized before executing the inference request and represents a key factor affecting responsiveness in elastic cloud deployments.

2) Throughput Metrics: To evaluate system scalability under concurrent workloads, we measure inference throughput using two complementary metrics. The first is *Requests Per Second (RPS)*, which indicates the number of inference requests processed per second by the system. The second is *Tokens Per Second (TPS)*, representing the number of output tokens generated per second during model inference. While RPS reflects system-level request handling capacity, TPS measures the efficiency of token generation during decoding.

3) Cost Efficiency Metrics: Because the proposed framework targets serverless and hybrid edge–cloud deployments, operational cost is a key evaluation criterion. We estimate the cost per request (CPR) and total operational expenditure (OPEX) using pricing models derived from a real-world GPU rental and serverless platform.

a) Cloud Cost Estimation: We model cloud inference cost based on GPU rental pricing using the closest matching GPU configuration to our server environment. Specifically, we use the hourly pricing of RunPod instances as a reference.

The cost per request under a dedicated GPU instance is approximated as:

$$C_{\text{cloud}} = \frac{P_{\text{gpu}} \cdot T_{\text{alloc}}}{N_{\text{req}}}, \tag{5}$$

where P_{gpu} (USD/hour) is the hourly rental cost of the GPU instance, T_{alloc} (hours) is the total time the instance is allocated for serving requests, and N_{req} is the total number of requests processed during that period.

To emulate a serverless operation cost, we consider RunPod Flex pricing with the same GPU specifications. In this case, the cost per request for serverless computation is calculated as:

$$C_{\text{serverless}} = P_{\text{flex}} \cdot T_{\text{exec}} + C_{\text{overhead}}, \tag{6}$$

where P_{flex} (USD/hour) is the serverless GPU rate, and C_{overhead} captures additional overhead costs such as container startup time and invocation latency. This formulation reflects the pay-per-use nature of serverless GPU inference.

b) Edge Cost Estimation: For edge devices, we follow an energy-based cost model. The cost per request is:

$$C_{\text{edge}} = E_{\text{query}} \cdot R_{\text{elec}} + \frac{C_{\text{device}}}{L}, \tag{7}$$

where E_{query} (kWh) is the energy consumption per inference, R_{elec} is the electricity price, C_{device} is the hardware cost, and L is the estimated total number of inferences over the device lifetime. This formulation reflects both operational energy cost and amortized hardware cost.

c) Total Operational Cost: Given a workload of W inference requests, the total cost is:

$$C_{\text{total}} = \sum_{i=1}^{W} C_i, \tag{8}$$

where C_i corresponds to either edge, dedicated cloud, or serverless execution depending on the placement decision.

D. Weak Models / SLMs

For weak models, we consider a diverse collection of recent 4-bit quantized SLMs and diffusion-based SLMs. These models are selected because they provide favorable trade-offs between computational efficiency, memory footprint, and reasoning capability, making them suitable for deployment on edge devices and resource-constrained environments. Furthermore, they represent different architectural paradigms, allowing us to evaluate the generalization capability of the proposed routing framework across both autoregressive and diffusion language models.

- **Gemma-4 Models**
  1) **Gemma-4-E2B-A4B**: A lightweight 2B-parameter instruction model optimized for low-latency inference with minimal memory consumption.
  2) **Gemma-4-E4B-A4B**: A 4B-parameter variant providing stronger reasoning capability while maintaining efficient deployment on consumer-grade GPUs.
  3) **Gemma-4-12B-A4B**: A larger SLM offering significantly improved reasoning and instruction-following performance while remaining considerably more efficient than frontier LLMs.
- **Phi-4-mini-3.8B**: Microsoft's compact language model designed for efficient reasoning and code generation, serving as a representative high-performance SLM.
- **Meta-Llama-3.1-8B-Instruct**: A widely adopted open-weight instruction-tuned model that serves as a strong baseline among current SLMs.
- **Diffusion SLMs**
  1) **LLaDA-MoE-7B-A1B-Instruct-Q4**: A diffusion-based mixture-of-experts language model demonstrating competitive reasoning performance while activating only a small subset of parameters during inference.
  2) **Dream-v0-Instruct-7B**: A diffusion language model from Huawei Noah's Ark Lab optimized for instruction-following tasks, with top-tier performance.
  3) **Fast-dLLM-v2-7B**: A recent block diffusion language model (dLLM) that efficiently adapts a pre-trained AR model (Qwen2.5-7B-Instruct) into a diffusion-style decoder for parallel text generation.

E. Strong Models

For strong models, we select recent open-weight large language models that provide substantially stronger reasoning capability than the selected SLMs while remaining deployable through Split-LLM inference. These models span different model scales and architectures, allowing the proposed router to learn when computationally expensive inference is justified. All models are deployed using 4-bit quantization to reduce GPU memory consumption during Split-LLM execution.

- **Qwen3-72B-Instruct**: Alibaba's flagship 72B instruction-tuned model with state-of-the-art multilingual reasoning, coding, and mathematical problem-solving capabilities.
- **Llama-3.3-70B-Instruct**: Meta's latest 70B instruction model offering strong general-purpose reasoning, long-context understanding, and competitive performance across a wide range of benchmark tasks.
- **NVIDIA Nemotron-3 Nano-30B**: NVIDIA's 30B reasoning-oriented language model optimized for efficient deployment while maintaining competitive performance on instruction-following and reasoning benchmarks. Its relatively smaller parameter count also enables comparison between medium-scale and very large Split-LLMs.

F. Baseline Routing Methods

To evaluate the effectiveness of the proposed architecture, we compare our approach against several recent query-level routing methods for collaborative SLM/LLM inference. Since the original baselines were developed under different model pools, task settings, and routing objectives, we slightly adapt their implementations to match our experimental context. In particular, we replace their original candidate models with the weak-model and strong-model sets defined in this study, standardize the input and output formats, and reformulate their routing objectives to select between an SLM and a stronger Split-LLM. Where necessary, we also modify the prediction heads, training labels, and cost functions while preserving the core routing mechanisms of the original methods. These adaptations enable a controlled and fair comparison under the same datasets, model pairs, quantization settings, and evaluation metrics.

The considered baselines are as follows:

- **RouteLMT** [14]: The original RouteLMT learns a lightweight in-model router from the prompt-token representations of a small translation model to estimate the expected performance gain from invoking a larger model before response generation. In our implementation, we replace the translation-specific backbone and objective with the selected SLMs and Split-LLMs. The router is trained to estimate the expected quality improvement achieved by escalating each query from the weak model to the strong model.

- **ICL-Router** [15]: The original method operates in two stages. First, queries are embedded and projected into compact vectors, with a projector and an LLM-based router trained to reconstruct the original queries and align the projected representations with the router's semantic space. Second, candidate models are profiled on a query set, and the router predicts their likelihood of correctly answering unseen queries using in-context representations of query characteristics and model performance. We adapt the model profiles to our weak and strong model sets and reformulate the final prediction as an SLM-versus-Split-LLM routing decision.
- **RouteLLM** [17]: RouteLLM trains a router to dynamically select between a weaker and a stronger language model during inference, thereby balancing response quality and inference cost. We retain its pairwise routing formulation but replace the original model pair and preference data with the SLM/Split-LLM pairs and benchmark-derived performance labels used in our study. Its cost-aware decision threshold is also calibrated according to the measured computational and communication costs of our deployment setting.
- **Arch-Router-1.5B** [18]: Arch-Router is an open-weight 1.5B-parameter routing model that maps queries to domain and action preferences for selecting an appropriate candidate model. We modify its original multi-model action space into the routing categories considered in our framework and fine-tune its routing head using the same weak-model and strong-model supervision employed by the proposed method. The underlying pretrained representation and domain-aware routing mechanism are otherwise preserved.

These modifications are limited to the components required for compatibility with our experimental setting. The principal representation-learning and routing strategies of each baseline remain unchanged, ensuring that the comparison reflects their core methodological characteristics rather than implementation differences.

VI. Results and Discussion

In this section, we show and discuss the experiment results ...

A. Overall System Latency

.

B. Cost Reduction

.

C. Research Limitations

.

VII. Conclusion and Future Work

The conclusion goes here.

References

[1] S. Zhang et al., "Understanding Users' Privacy Perceptions Towards LLM's RAG-based Memory", in Proceedings of the 2025 Workshop on Human-Centered AI Privacy and Security, 2025, pp. 10–19.

[2] F. He, T. Zhu, D. Ye, B. Liu, W. Zhou, and P. S. Yu, "The Emerged Security and Privacy of LLM Agent: A Survey with Case Studies", ACM Comput. Surv., vol. 58, no. 6, Dec. 2025.

[3] S. Ostrovskiy, "Best practices for HIPAA compliance of large language models (LLMs)", 10-Dec-2025. https://edenlab.io/blog/hipaa-compliant-ai-best-practices (accessed Mar 31, 2026).

[4] E. Ford et al., "What is the patient re-identification risk from using de-identified clinical free text data for health research?", AI Ethics, vol. 5, no. 5, pp. 4441–4454, Feb. 2025.

[5] N. D. Pham, K. T. Phan, A. Abuadbba, Y. Gao, V. -D. Nguyen and N. Chilamkurti, "Split Learning Without Local Weight Sharing to Enhance Client-Side Data Privacy," in IEEE Transactions on Dependable and Secure Computing, vol. 22, no. 5, pp. 5352-5365, Sept.-Oct. 2025, doi: 10.1109/TDSC.2025.3565229.

[6] G. S. Hukkeri, R. H. Goudar, G. M. Dhananjaya, V. N. Rathod and S. Ankalaki, "A Comprehensive Survey on Split-Fed Learning: Methods, Innovations, and Future Directions," in IEEE Access, vol. 13, pp. 46312-46333, 2025, doi: 10.1109/ACCESS.2025.3547641.

[7] A. Alhindi, S. Al-Ahmadi and M. Maher Ben Ismail, "Balancing Privacy and Utility in Split Learning: An Adversarial Channel Pruning-Based Approach," in IEEE Access, vol. 13, pp. 10094-10110, 2025, doi: 10.1109/ACCESS.2025.3528575.

[8] A. Patra, S. Kundu, A. Dey, T. Saha, S. Karmakar and S. K. Bishnu, "A Survey on On-Device Small Language Models: Architectures, Compression, Applications, and Open Challenges," 2026 9th International Conference on Electronics, Materials Engineering & Nano-Technology (IEMENTech), Kolkata, India, 2026, pp. 1-6, doi: 10.1109/IEMENTech202669403.2026.11434122.

[9] F. Wang et al., "A Survey on Small Language Models in the Era of Large Language Models: Architecture, Capabilities, and Trustworthiness", in Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2, Toronto ON, Canada, 2025, pp. 6173–6183.

[10] P. Mai, R. Yan, Z. Huang, Y. Yang, and Y. Pang, "Split-and-Denoise: Protect large language model inference with local differential privacy", arXiv preprint arXiv:2310. 09130, 2023.

[11] W. Zheng et al., "CITER: Collaborative Inference for Efficient Large Language Model Decoding with Token-Level Routing", arXiv preprint arXiv:2502. 01976, 2025.

[12] C. Zheng, K. Zhang, W. Zhang, Q. Liu, A. A. Tesfay, and Others, "Fast Collaborative Inference via Distributed Speculative Decoding", Journal of Information and Intelligence, 2026.

[13] Y. Xu, S. Zhou, and Z. Niu, "DiP-SD: Distributed Pipelined Speculative Decoding for Efficient LLM Inference at the Edge", arXiv preprint arXiv:2604. 20919, 2026.

[14] Y. Luo et al., "RouteLMT: Learned Sample Routing for Hybrid LLM Translation Deployment", 2026.

[15] C. Wang et al., "ICL-Router: In-Context Learned Model Representations for LLM Routing", in Proceedings of the AAAI Conference on Artificial Intelligence, 2026, vol. 40, pp. 33413–33421.

[16] W. Song et al., "IRT-Router: Effective and Interpretable Multi-LLM Routing via Item Response Theory", in Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2025, pp. 15629–15644.

[17] I. Ong et al., "RouteLLM: Learning to Route LLMs from Preference Data", in International Conference on Learning Representations, 2025, vol. 2025, pp. 34433–34448.

[18] C. Tran, S. Paracha, A. Hafeez, and S. Chen, "Arch-Router: Aligning LLM Routing with Human Preferences", arXiv preprint arXiv:2506. 16655, 2025.

[19] J. Wei et al., "Measuring short-form factuality in large language models", arXiv preprint arXiv:2411. 04368, 2024.

[20] H. Li et al., "LLMRouterBench: A Massive Benchmark and Unified Framework for LLM Routing", in Findings of the Association for Computational Linguistics: ACL 2026, 2026, pp. 37733–37754.